

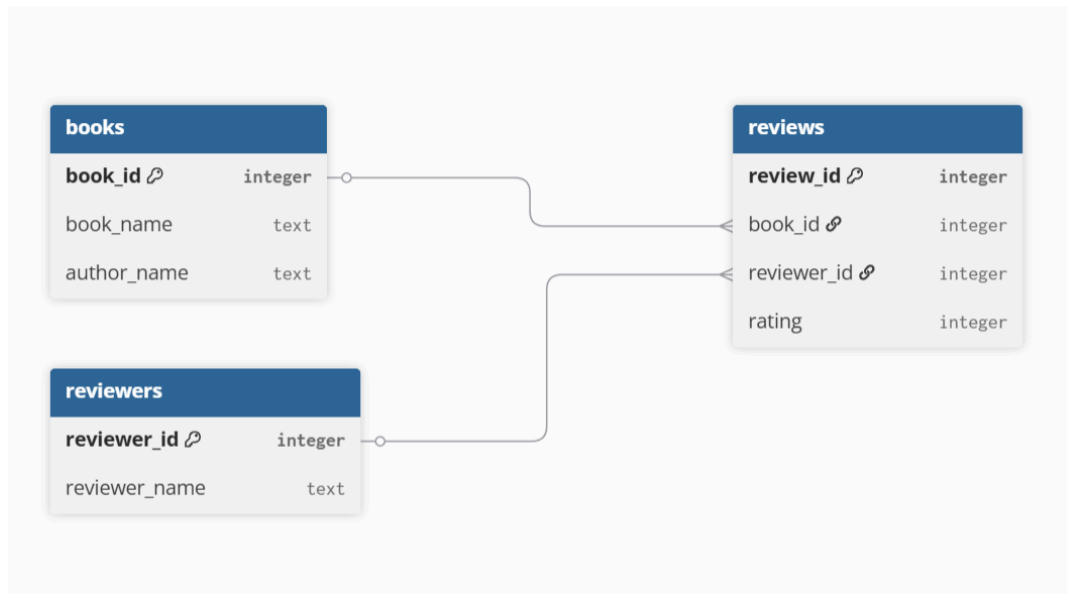
Introduction

*When deciding on my topic of choice for my Database project, I realized that although we have millions of books available to us, it's still quite difficult to find a book that actually resonates with our personal taste. We are often faced with the problem of the "paradox of choice". This is a problem that I run into often as an avid reader, and I end up choosing books based on author, genre, or the hottest/trending ones occurring at the time. Additionally, recommendation tools that are available only rely on global averages, which fail to capture the essence of an individual's preferences. Originally, the goal of this project was to solve this problem by building a search engine discovery tool and intended to serve as a broad social platform where you can type in a vibe/feeling you would want in a book, and that recommendation would pop up. This involved using a complex mix of Vector databases and NoSQL called the "R-Reader Engine" to capture community memory from group notes and book clubs. I wanted to solve this problem by using fragmented data and being a predictor for genre burnout through emotional tone scoring and genre burnout.

*However, as I started working in the development phase of this project, it came to my attention that I needed to make changes in the direction it was headed. In the timeframe between the proposal and when this project was due, the social-heavy platform was too broad and risked making the data hard to manage and making the data messy. Instead of a search engine, I decided to pivot towards building a relational SQL foundation that would provide high-performance recommendations from this engine. By making this pivot, I was able to turn my focus towards data accuracy, mathematical vectors used for cosine similarity, and it's ability to be a predictive analyzer rather than a search engine tool. This felt best for the objectives of this project and course as we have learned a lot about structured relationships between entities in relational database schema, while serving to be professionally robust and technically attainable. The final product for this project is now called the Vector Verse.

Database Design and Cloud-Based Integration

*A major technical goal that I had for this project was to ensure that the application that was built has the ability to remember users and their ratings across different devices and sessions. In order to make this happen, I built a cloud-hosted infrastructure and moved away from using local files. This was done by building a relational PostgreSQL database hosted on Neon, where I cleaned and gathered large datasets from Kaggle and my own manual search from sites such as Amazon, Goodreads, and Reddit for the library. With large volumes of data, it is easy for it to quickly become unorganized; therefore, I split it into three tables: Reviewers, Reviews, and Books. The Reviewers table acts as a directory for user names, the reviews table acts as the connection between users to the books they have read, and lastly, the books table serves as a library catalog. Efficient querying and data integrity were achieved through this structure.



*A technique that I used to make sure the system works correctly for multiple people at once is called Atomic transactions. It was used through the dbWithTransaction function and ensures that the system treats two actions as a single move when a user adds a new book and rating at the same time. If the system crashes or an error happens halfway through, it cancels the whole move rather than saving half of it. As we have learned in class, this satisfies transactions of ACID properties: atomicity (process must happen in its entirety or not at all), consistency, isolation, and durability. With this transaction installed, data that has a rating floating around with no book attached to it wouldn't occur. The key to professional database management is that the library stays reliable and clean when users access the app and add their own data.

Book Recommender

All OK Ask AI Upgrade R

Tables
production

neondb Database studio

public

Search...

books reviews reviews

book_id serial book_name text author_name text reviews

50 rows • 187ms

1	A Court of Mist and Fu...	Sarah J. Maas	reviews
2	A Court of Thorns and ...	Sarah J. Maas	reviews
3	A Court of Wings and R...	Sarah J. Maas	reviews
4	A Little Life	Hanya Yanagihara	reviews
5	All the Light We Canno...	Anthony Doerr	reviews
6	Atomic Habits	James Clear	reviews
7	Balloons over Broadway	Melissa Sweet	reviews
8	Brown Bear, Brown Bear...	Bill Martin Jr. & Eric...	reviews
9	Chicka Chicka Boom Boom	Bill Martin Jr. & John...	reviews
10	Demon Copperhead	Barbara Kingsolver	reviews
11	Dirty Thirty (Stephani...	Janet Evanovich	reviews
12	Don't Let the Pigeon D...	Mo Willems	reviews
13	Elon Musk	Walter Isaacson	reviews
14	First Little Readers P...	Deborah Schecter	reviews
15	Five Silly Turkeys	Salina Yoon	reviews
16	Flash Cards: Sight Wor...	Scholastic Teacher Res...	reviews
17	Forever Strong	Dr. Gabrielle Lyon	reviews
18	Fourth Wing	Rebecca Yarros	reviews
19	Friends, Lovers, and t...	Matthew Perry	reviews
20	From Crook to Cook	Snoop Dogg	reviews
21	Goodnight Moon	Margaret Wise Brown	reviews
22	Goodnight, Goodnight C...	Sherri Duskey Rinker	reviews
23	Harry Potter and the P...	J.K. Rowling	reviews

Book Recommender

All OK Ask AI Upgrade R

Tables

production

neondb

Database studio

public

Search...

books

reviewers

reviews

reviewer_id	reviewer_name	reviews
1	*AT	reviews
2	A H Kobayashi	reviews
3	a reader	reviews
4	A Reviewer	reviews
5	A. K. P.	reviews
6	A. Slater	reviews
7	A.B.318	reviews
8	A.S.	reviews
9	a_bennett	reviews
10	AB1213	reviews
11	Abby Dye	reviews
12	Abigail	reviews
13	acpatrick	reviews
14	Adam	reviews
15	Adam Manning	reviews
16	Adam Nelson	reviews
17	Adam the book reader	reviews
18	Adrea Crosby	reviews
19	Agnes Shapiro	reviews
20	Aidan Longueville	reviews
21	aiden	reviews
22	Aimee	reviews

50 rows • 184ms

Book Recommender

All OK Ask AI Upgrade R

Tables

production

neondb

Database studio

public

Search...

books

reviewers

reviews

review_id	book_id	reviewer_id	rating	book	reviewer
1	26	1	5	book	reviewer
2	39	1	3	book	reviewer
3	14	1	5	book	reviewer
4	60	1	2	book	reviewer
5	42	1	2	book	reviewer
6	89	1	2	book	reviewer
7	52	1	4	book	reviewer
8	85	1	3	book	reviewer
9	21	1	3	book	reviewer
10	44	1	4	book	reviewer
11	6	2	4	book	reviewer
12	36	2	4	book	reviewer
13	46	2	1	book	reviewer
14	3	2	1	book	reviewer
15	30	2	2	book	reviewer
16	14	2	1	book	reviewer
17	7	2	5	book	reviewer
18	22	2	4	book	reviewer
19	60	2	1	book	reviewer
20	15	2	5	book	reviewer
21	82	3	4	book	reviewer
22	32	3	1	book	reviewer

50 rows • 176ms

Application

*In the development of using R and Shiny, I encountered a technical obstacle. When extracting book review data from sources such as Goodreads, Amazon, and Reddit, I used as much authentic data from what I had found and incorporated it into the engine to support real-world applicability, but faced a limitation in that a majority of reviewers had only contributed to one or two ratings. In these beginning stages of analysis, which were revealed in the dataset, there was a gap in order for the recommendation system to work because it relies on behavioral similarities between individuals, and not having a big enough ratio between reviewers and ratings created a lack of data density. There is no mathematical bridge if a user has only rated a single book or any history that can connect their book preferences to the rest of the community, making it much harder for the Cosine Similarity algorithm to work. With this initial issue, the Cosine Similarity that I wanted to implement could not draw meaningful conclusions.

*To overcome this technical obstacle, I developed a pseudo-data pipeline to demonstrate a more established long-term user base. Throughout this process, I kept existing book titles and reviewers and added additional randomized ratings to fill in the gaps in order to increase the robustness of the dataset. This was a vital step to implement in order to allow a mathematical model to look past basic metadata and for it to actually analyze behavioral patterns instead. It provides a healthier history of reviews and simulates a more mature system. For a recommendation system to be useful, there needs to be a sufficient data set for it to be useful for the users. By adding a more robust data structure, the system can now provide personalized recommendations by patterns of preference that are mathematically backed up.

* The front-end architecture of the application is built on the Shiny framework on the principles of reactive programming. Since implementing Shiny is a part of the project requirement, a specific challenge arose because the application is connected to the Neon cloud database. The issue with Shiny is that it is not designed to automatically detect updates or changes made by other users in a remote environment. Because of this problem, I implemented a custom refresh trigger logic. When a user registers a new account or submits a new rating, it forces the application to requery the database and ensures that information that pops up on the interface is an accurate reflection of what the current cloud state is, regardless of when the data was last pulled. The user interface utilizes a sidebar to main panel layout to guide the user within the book discovery process. To help with this process, I also employed a specific coding logic for when a user is successfully signed in, then the registration and logic boxes are automatically hidden. I wanted to reduce any visual clutter so that users can get the full experience of the core purpose of the app, which is reviewing their personal reading history and exploring their own personalized recommendations.

Book Recommendation System

Login Here:

Create New Profile...

Your Name:

Sign Up

Rate a Book

Title:

Author:

Rating (1-5):

1

2

3

4

5

Welcome to the Book Engine!

Personalized Picks

Your History

Study Expectations

The core "brain" of the R-Reader Engine recommendation logic is built on Collaborative Filtering using the Cosine Similarity metric. This sophisticated approach identifies deep relationships between users based on their shared behavioral patterns—specifically their historical rating activity—rather than relying solely on static book metadata like genres or publication dates. The fundamental idea behind this method is that if two users agree on the quality of several books in the past, they are highly likely to agree on others in the future. To move this concept from theory to a functional engine, the relational data stored in the PostgreSQL reviews table must undergo a significant transformation. The system converts these raw lists of reviews into a high-dimensional User-Item Matrix, where every unique reader is represented as a row and every individual book is represented as a column.

In this mathematical coordinate system, a user's entire taste profile is treated as a single, complex vector signature. Essentially, every person is represented as an arrow pointing in a specific direction within a multi-dimensional preference space. The algorithm determines the similarity between users by calculating the cosine of the angle between these respective vectors using the formal equation:

$$\text{Similarity (A, B) = Cosine} = \frac{A \cdot B}{\|A\| \|B\|}$$

* The recommendation logic is built on collaborative filtering using the Cosine Similarity metric. In the

By focusing on the orientation and angle of these "arrows" rather than their raw length or magnitude, the engine achieves a critical level of normalization. This is a vital distinction for rating data because it accounts for individual user bias and different rating styles. For example, a "harsh" rater who rarely gives a score above a three and a "generous" rater who frequently gives fives can still be identified as mathematically similar "Book Twins" if they like and dislike the same books relative to one another. The math recognizes that their vectors are pointing in the same geometric direction, even if one vector is "longer" than the other due to higher raw scores.

The primary distinction between the R-Reader Engine and a basic matching system is this use of high-dimensional pattern recognition. While two people both giving a "5" to a single popular bestseller might be a mere coincidence, Cosine Similarity evaluates the entire landscape of their shared library to ensure a genuine behavioral alignment. Because every book in the database acts as a unique dimension, a user's preference is not just a single data point, but a pattern that must align across multiple axes. For two users to be officially matched, they must share a consistent pattern of liking and disliking the same sets of books. Furthermore, the mathematical structure of the equation includes a normalization component in the denominator that prevents "high-volume" raters from skewing the results. By dividing the shared ratings (the dot product) by the total magnitude of each user's history, the system ensures that the proportional pattern of taste is prioritized over the raw volume of books read. This prevents a user who gives every book a "5" from accidentally matching with the entire community by default.

To prevent the engine from making wild or inaccurate guesses, a "shared bridge" logic was implemented into the application code. This safeguard identifies specific books that have been rated by at least two different people in the database. If a new user joins and has not yet rated any of these "shared" books, the app politely requests that they rate a community favorite first. This ensures the similarity math has a solid, verified starting point for comparison and effectively avoids the "Cold Start" problem commonly found in new recommendation systems. To further ensure the engine provided immediate value during the testing

phase, the database was pre-seeded with hundreds of historical reviews and supplemented with custom pseudodata to increase the density of the User-Item Matrix.

Once the similarity scores (ranging from 0 to 1) are calculated for every user in the database, the system identifies the Top 10 Nearest Neighbors for the active user. These individuals represent the readers whose historical tastes most closely align with the user's current profile. The engine then performs a final scan of the titles that these neighbors rated highly, filters out any books the active user has already encountered, and ranks the remaining titles by their average neighbor score. The final result is a display of the Top 5 Picks, ensuring that the recommendations are based on mathematically verified common ground rather than superficial or isolated data points. Testing revealed that this engine is highly sensitive to extreme ratings, meaning that a single 1-star or 5-star review can significantly shift the direction of a user's vector and change who the system identifies as their most compatible reading twin.

Future Scaling

The final application was successfully hosted on shinyapps.io, moving the project from a local script to a production server. This move validated the application's ability to operate as a true web service. It proved that the R script could successfully manage a secure connection to the cloud database while serving multiple users across different geographic locations simultaneously. Looking toward the future, there are several ways to make this engine even more professional. As the library grows to thousands of books, the current math will start to slow down because it uses a dense matrix approach. I plan to switch to a more efficient way of storing data using sparse matrices, which only store actual ratings and ignore the empty space to save memory.

I also want to implement hybrid intelligence, which would incorporate content-based filtering. By scraping metadata like genres or tags, the app could recommend books by the same author even if no other users have rated them yet. Another important feature for scaling is the introduction of entropy-based alerts. By monitoring the variance in a user's recent ratings, the engine could detect declining interest and automatically suggest a palate cleanser from a completely different category to keep the experience fresh. Finally, implementing asynchronous processing would allow the heavy math calculations to run in the background, preventing the interface from freezing while the engine finds matches. This transition from the initial social proposal to the current specialized system ultimately resulted in a more scalable and professional application that demonstrates high-level technical maturity and database administration skill.

Screenshots for paper

```
2 v CREATE TABLE books (  
3     book_id SERIAL PRIMARY KEY,  
4     title VARCHAR(255) NOT NULL,  
5     author VARCHAR(100),  
6     genre VARCHAR(50),  
7     description TEXT,  
8     published_year INTEGER  
.  
  
2 v CREATE TABLE reviewers (  
3     reviewer_id SERIAL PRIMARY KEY,  
4     reviewer_name TEXT
```

```

15 v CREATE TABLE reviews (
16     review_id SERIAL PRIMARY KEY,
17     book_id INT REFERENCES books(book_id),
18     reviewer_id INT REFERENCES reviewers(reviewer_id),
19     rating INT CHECK (rating >= 1 AND rating <= 5)

```

```

1 v SELECT b.book_name, COUNT(r.review_id) as total_reviews, AVG(r.rating) as avg_score
2 FROM books b
3 JOIN reviews r ON b.book_id = r.book_id
4 GROUP BY b.book_name
5 ORDER BY avg_score DESC
6 LIMIT 10;

```

```

1 v SELECT
2     b.book_name,
3     b.author_name,
4     r.rating
5 FROM reviews r
6 JOIN books b ON r.book_id = b.book_id
7 JOIN reviewers rev ON r.reviewer_id = rev.reviewer_id
8 WHERE rev.reviewer_name = 'Richa Gandhi';

```

```

1 v SELECT r.review_id, v.reviewer_name, b.book_name, r.rating
2 FROM reviews r
3 JOIN reviewers v ON r.reviewer_id = v.reviewer_id
4 JOIN books b ON r.book_id = b.book_id
5 WHERE r.reviewer_id > 813;

```

Book Recommendation System

Login Here:

Create New Profile...

Your Name:

Sign Up

Rate a Book

Title:

Author:

Rating (1-5):

1

2

3

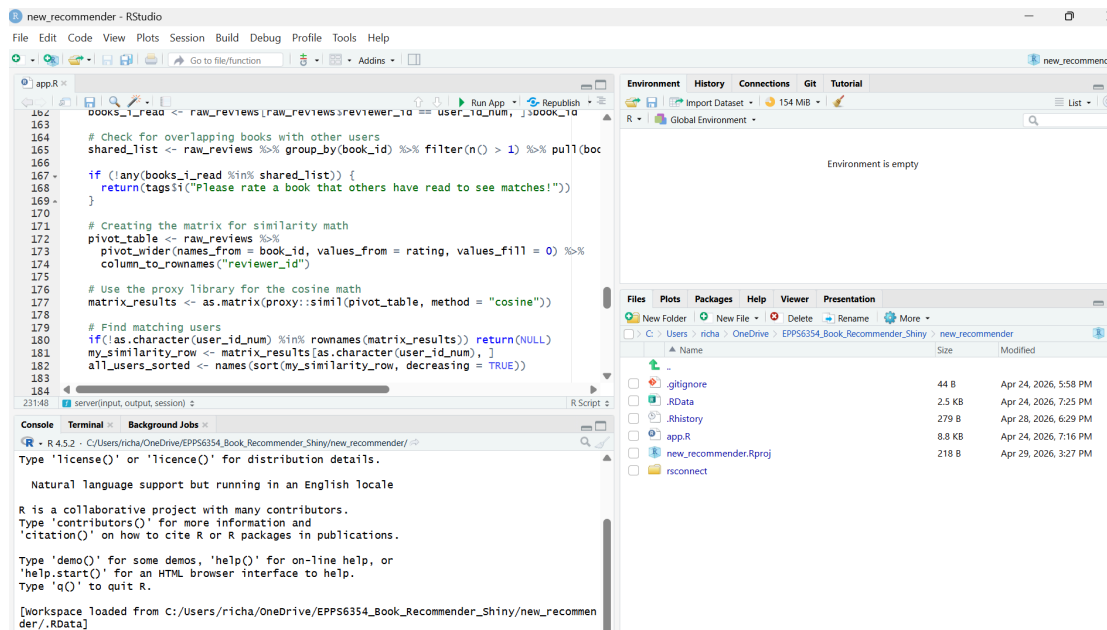
4

5

Welcome to the Book Engine!

Personalized Picks

Your History



AI disclosure

AI-based tools, including ChatGPT and Google Gemini, were used solely for conceptual clarification during this database project and helped with the creation of the name “Vector Verse”. They were not used in the writing of this paper or in the development of the project idea. All design decisions, schema development, and technical implementation were completed based on my own understanding and effort. These tools served only as an aid and did not replace my original work or analysis.